Python Debugging Quiz

Spot the Bug

Instructions

Each question presents a Python program (or pair of files) that contains **exactly one bug**. The bug may cause a runtime error, a wrong result without any error, or an unexpected crash on a second call. Your task is to identify the bug and circle it in the code.

Question 1

This gradebook calculator runs without errors but consistently produces wrong output. The bug does not raise any exception.

```
# Gradebook average calculator

def calculate_average(scores):
    if not scores:
        return 0.0
    total = 0
    for score in scores:
        total += score
    return total / len(scores)

def get_grade(average):
    if average ≥ 90:
        return "A"
    elif average ≥ 80:
        return "B"
    elif average ≥ 70:
        return "C"
    else:
        return "F"

def process_students(student_data):
    results = {}
    for name, scores in student_data.items():
        avg = calculate_average(scores)
        results[name] = {
            "average": avg,
            "grade": get_grade(avg)
        }
    return results

def print_report(results):
    list = sorted(results.keys())
    for name in list:
        data = results[name]
        print(f"{name}: avg={data['average']:.1f}, grade={data['
            grade']}")
    print("---")
    averages = [r["average"] for r in results.values()]
    top = max(averages)
    print(f"Top average: {top:.1f}")
    print(f"Class size: {list(results.keys())}")

students = {
    "Alice":  [88, 92, 79, 95],
```

```
    "Bob":    [72, 68, 74, 70],
    "Carol":  [95, 98, 100, 97],
}

report = process_students(students)
print_report(report)
```

Question 2

This inventory manager crashes with a `RuntimeError` part-way through cleanup. The logic looks correct at first glance.

```
# Warehouse inventory cleanup

def load_inventory():
    return {
        "apple":      120,
        "banana":       0,
        "cherry":      45,
        "date":         0,
        "elderberry":   8,
        "fig":          0,
        "grape":      200,
    }

def remove_empty(inventory):
    # Remove items with zero stock
    for item in inventory:
        if inventory[item] == 0:
            del inventory[item]
    return inventory

def apply_restock(inventory, restock):
    for item, qty in restock.items():
        if item in inventory:
            inventory[item] += qty
        else:
            inventory[item] = qty
    return inventory

def low_stock_alert(inventory, threshold=10):
    alerts = []
    for item, qty in inventory.items():
        if qty < threshold:
            alerts.append(f"{item}: only {qty} left")
    return alerts

inv = load_inventory()
inv = remove_empty(inv)

restock = {"elderberry": 50, "kiwi": 30}
inv = apply_restock(inv, restock)

alerts = low_stock_alert(inv)
for alert in alerts:
    print(alert)
```

Question 3

A shopping cart system split across two files. Adding items works, but totals are always wrong and per-product discounts are never applied. No exceptions are raised.

File: `product.py`

```python
class Product:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category
        self.discounted_price = price

    def apply_discount(self, pct):
        self.discounted_price = self.price * (1 - pct / 100)

    def __repr__(self):
        return f"Product({self.name},   {self.price})"
```

File: `cart.py`

```python
from product import Product

class Cart:
    def __init__(self):
        self.items = []
        self.discount_pct = 0

    def add(self, product, qty=1):
        self.items.append((product, qty))

    def apply_discount(self, pct):
        self.discount_pct = pct

    def total(self):
        subtotal = 0
        for product, qty in self.items:
            subtotal += product.price * qty
        factor = 1 - self.discount_pct / 100
        return subtotal * factor

    def receipt(self):
        for p, qty in self.items:
            print(f"  {p.name} x{qty} @   {p.discounted_price:.2f
                } each")
        print(f"  Total:   {self.total():.2f}")

milk  = Product("Milk",  1.20, "dairy")
bread = Product("Bread", 2.50, "bakery")
wine  = Product("Wine", 12.99, "alcohol")
```

```
29
30 cart = Cart()
31 cart.add(milk,  3)
32 cart.add(bread, 2)
33 cart.add(wine,  1)
34
35 # Apply 10% off wine and 20% cart discount
36 wine.apply_discount(10)
37 cart.apply_discount(10)      # line 37  (comment says 20%)
38
39 cart.receipt()
```

Question 4

A student split their code into two files. The program crashes immediately on run.

File: `math_utils.py`

```
def square(n):
    return n ** 2

def cube(n):
    return n ** 3

def is_perfect_square(n):
    if n < 0:
        return False
    root = int(n ** 0.5)
    return root * root == n

def factors(n):
    result = []
    for i in range(1, int(n ** 0.5) + 1):
        if n % i == 0:
            result.append(i)
            if i ≠ n // i:
                result.append(n // i)
    return sorted(result)
```

File: `main.py`

```
from math_utils import square

result = cube(4)
print(result)

print(is_perfect_square(16))
print(factors(12))
```